

Part 0: Training infrastructure and sanity checks

Deliverables (as required by the assignment)

We provide the following for Part 0 and Part 0.1:

- **Code:** `train.py` (config-driven training loop, metrics logging, checkpoints), `inference.py` (load checkpoint and greedy generation), `model.py` (GPT with optional padding mask for batched sequences), `tokenizer_utils.py`, and autograder stubs in `part_0_1_contract.py`.
- **Experiment configs (YAML):** `configs/sanity.yaml`, `configs/sanity_quick.yaml`, `configs/sanity_suffix_quick.yaml` (hyperparameters, task type, output directory, random seed, and loss-masking options are all recorded in the config and copied to `config_resolved.yaml` under each run).
- **Logs:** per run, `metrics.csv` with training/validation/test loss and accuracy at evaluation steps.
- **Checkpoints:** `ckpt_<step>.pt` (model weights, `GPTConfig`, resolved training config, step index) and `tokenizer.json` saved in the run directory (e.g. `runs/sanity_quick_test/`, `runs/sanity_suffix_quick/`).

Sanity check 1: memorization

We train a **single-layer** transformer on a dataset consisting of one line: `I love machine learning`, with BOS/EOS as in `CharTokenizer`. We train on the full sequence (all non-padding next-token positions contribute to the loss). After optimization, **training loss goes to 0** and **greedy sampling from BOS alone** reproduces the string.

Reproduction.

```
python train.py --config configs/sanity_quick.yaml
python inference.py --checkpoint_dir runs/sanity_quick_test --prompt "" --max_new_tokens 64
```

The second line should print the target sentence (with BOS/EOS in the first line; the “skip specials” decode should match `I love machine learning` exactly).

Sanity check 2: masked prefix (suffix prediction)

We use the same string but **mask the loss on the first three body tokens** after BOS (i.e. we do not train on the predictions of `I`, the space, and `l`). The model is trained to predict only the remaining suffix. Evaluating with BOS alone is not the right test; we **condition on the prefix** `I l` (BOS is added by the script) and generate the rest.

Reproduction.

```
python train.py --config configs/sanity_suffix_quick.yaml
python inference.py --checkpoint_dir runs/sanity_suffix_quick --prompt "I l" --max_new_tokens 64
```

Training loss on the supervised positions should go to 0; the decode with the prefix above should again yield `I love machine learning`.

Modifications to the provided codebase

The base `model.py` follows the course/nano-GPT-style GPT. We extended the forward pass to accept an optional `key_padding_mask` so that padded batch positions are ignored in attention (as suggested in the handout for variable-length batches). The causal lower-triangular mask is always registered (not only in the non-flash path) so the same buffer can be combined with padding when using scaled dot-product attention. The AdamW construction in `configure_optimizers` must return the optimizer (used by the training script).

Training uses YAML configs, copies the resolved config to the run folder, sets Python/PyTorch RNG seeds from the config, and logs CSV metrics. Inference loads the latest `ckpt_*.pt` and `tokenizer.json` from a run directory.

Challenges

- Aligning **loss masks** with next-token prediction indices (shifted cross-entropy) for answer-only arithmetic and for sanity suffix masking.
- Ensuring **padding and attention** interact correctly so padded keys do not receive attention and training remains stable.
- For the suffix experiment, **sampling protocol**: full-string regurgitation from BOS is only expected when the full sequence is trained; with prefix masking, generation must be checked from the appropriate fixed prefix.